TP2 – Rotas na Mina: Utilizando os paradigmas (GULOSO e DIVISÃO E CONQUISTA)

Thaylor Weslei Dias G. Verteiro
Matrícula: 2023028668
Universidade Federal de Minas Gerais (UFMG) – DCC

Novembro de 2026

1 Introdução e Modelagem do Problema

Este trabalho aborda o problema de navegação autônoma de robôs em uma mina caracterizada por uma geometria estreita e extensa (poucas galerias paralelas e um número elevado de setores). A mina é modelada como uma grade bidimensional de dimensão $H \times W$, onde H representa o número de linhas ($2 \leq H \leq 5$) e W representa o número de colunas ($2 \leq W \leq 10.000$). Cada célula (i, j) possui um custo de travessia não negativo $c_{i,j}$ ($0 \leq c_{i,j} \leq 10^6$).

O mapa foi estruturado formalmente como um grafo ponderado nos vértices $G = (V, E)$:

- **Vértices (V):** Cada célula (r, c) corresponde a um nó indexado linearmente por $id = r \cdot W + c$, totalizando $|V| = H \cdot W$ vértices.
- **Arestas (E):** Conexões direcionadas entre células adjacentes na grade (norte, sul, leste e oeste), com grau máximo por nó igual a 4. Logo, $|E| \leq 4 \cdot H \cdot W$.
- **Custo do Caminho:** O custo total para transitar entre duas células equivale à soma dos pesos de todas as células visitadas ao longo do percurso, incluindo a origem e o destino. Para consultas em que a origem coincide com o destino, o custo é o peso da própria célula.

A central de operações precisa responder a um conjunto de Q consultas independentes ($1 \leq Q \leq 50.000$). Para isso, implementaram-se e compararam-se duas abordagens: uma baseada exclusivamente no paradigma guloso e outra fundamentada no paradigma de Divisão e Conquista.

2 Abordagem Gulosa (Dijkstra Direto)

A solução de referência emprega o algoritmo de Dijkstra com fila de prioridades (*min-heap*) aplicado individualmente para cada uma das Q consultas lidas da entrada.

Para responder a uma consulta entre (r_1, c_1) e (r_2, c_2), o algoritmo inicializa a distância da origem com seu próprio peso, $dist[start] = grid[r_1][c_1]$, e expande os nós de menor custo acumulado até atingir o nó de destino (parada antecipada).

2.1 Complexidade Teórica

- **Complexidade de Tempo:** Para cada consulta, a execução do Dijkstra sobre o grafo com $|V| = H \cdot W$ vértices consome $O(|V| \log |V|)$ no pior caso. Como as Q consultas são processadas de forma independente e sem reaproveitamento de cálculos anteriores, a complexidade de tempo total é $O(Q \cdot H \cdot W \log(H \cdot W))$.
- **Complexidade de Espaço:** A estrutura requer armazenamento da grade e do vetor de distâncias, resultando em $O(H \cdot W)$.

3 Abordagem por Divisão e Conquista

A solução otimizada explora a geometria $H \ll W$ particionando recursivamente o conjunto de colunas da mina.

Dada uma subgrade definida pelo intervalo de colunas $[L, R]$, seleciona-se a coluna central $m = \lfloor (L + R)/2 \rfloor$. A partir de cada uma das H células da coluna central, executa-se o algoritmo de Dijkstra restrito à faixa de colunas $[L, R]$.

A combinação dos caminhos para uma consulta entre u_1 e u_2 é dada por:

$$\text{custo_ponte} = \text{dist}_m[k][u_1] + \text{dist}_m[k][u_2] - c_{k,m} \tag{1}$$

Para cada consulta ativa na faixa, calcula-se o menor custo que cruza a coluna m testando todas as H células (k, m) como ponto de travessia intermediário. As consultas que estão estritamente contidas à esquerda ($c_1, c_2 < m$) ou à direita ($c_1, c_2 > m$) são filtradas e enviadas para as chamadas recursivas correspondentes, permitindo atualizar a menor distância encontrada.

3.1 Complexidade Teórica

- **Complexidade de Tempo:** A árvore de recursão possui profundidade $O(\log W)$. Em cada nível, são executados H Dijkstras em subgrades de largura total W, consumindo $O(H^2 \cdot W \log W)$ por nível. O roteamento das Q consultas ao longo da árvore consome $O(Q \log W)$. Assim, a complexidade final de tempo é $O(H^2 \cdot W \log W + Q \log W)$.
- **Complexidade de Espaço:** Armazenamento da matriz, vetor de consultas e respostas em $O(H \cdot W + Q)$.

4 Resultados Experimentais

Os experimentos foram conduzidos executando as duas implementações compiladas com a flag de otimização `-O2` contra o conjunto oficial de 31 casos de teste fornecidos na disciplina. As medições de tempo de CPU em segundos foram coletadas com precisão de milissegundos.

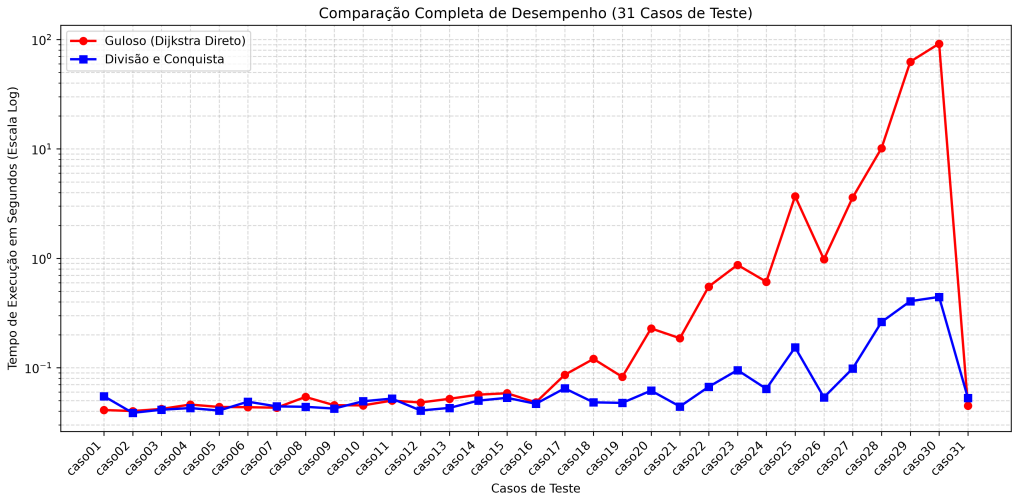


Figura 1: Comparação do tempo de execução em escala logarítmica entre a abordagem Gulosa e a Divisão e Conquista nos 31 casos de teste.

Tabela 1: Comparação de tempo de execução real (em segundos) variando W e Q ($H = 5$ fixo).

Caso	Colunas (W)	Consultas (Q)	Guloso (s)	Divisão e Conquista (s)
caso01	2	1	0,0411	0,0550
caso03	5	10	0,0420	0,0413
caso05	10	20	0,0438	0,0406
caso08	20	40	0,0541	0,0439
caso10	30	60	0,0453	0,0494
caso13	100	200	0,0520	0,0429
caso15	200	300	0,0584	0,0531
caso17	400	500	0,0860	0,0648
caso20	1.000	1.000	0,2287	0,0618
caso23	2.500	2.500	0,8707	0,0951
caso25	4.000	4.000	3,6964	0,1534
caso28	8.000	10.000	10,1544	0,2617
caso29	10.000	25.000	62,7070	0,4059
caso30	10.000	50.000	91,6369	0,4438

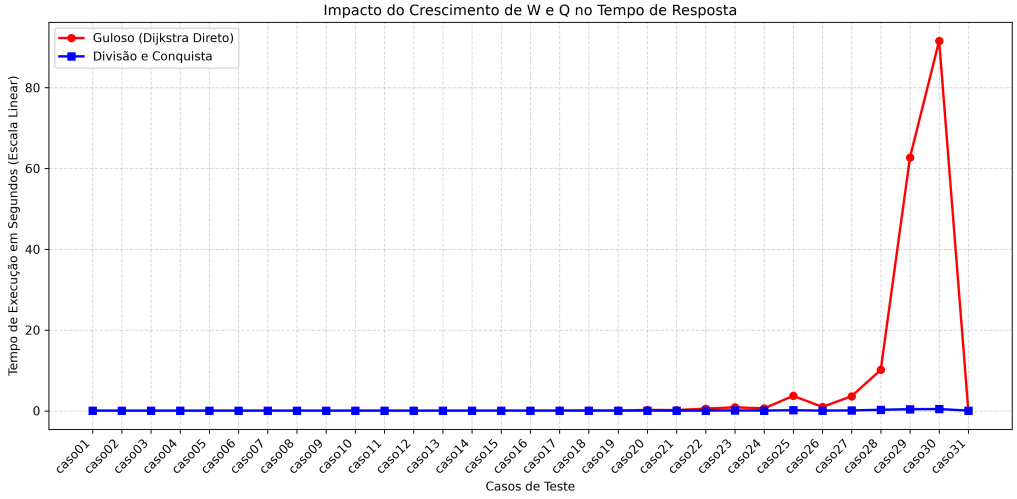


Figura 2: Impacto do crescimento de W e Q no tempo de resposta em escala linear.

5 Discussão dos Resultados

A análise dos dados empíricos valida categoricamente a superioridade assintótica da Divisão e Conquista. Para cenários reduzidos (como os casos 01 ao 12, onde $W \leq 80$ e $Q \leq 120$), ambas as soluções apresentam tempos equivalentes na faixa de 0,04 a 0,05 segundos. Nesses cenários pequenos, a sobrecarga de chamadas recursivas da Divisão e Conquista se aproxima do custo de poucos Dijkstras isolados.

Contudo, conforme W e Q aumentam, a curva de tempo do algoritmo Guloso cresce drasticamente. No **caso30** (onde $W = 10.000$ e $Q = 50.000$), o algoritmo Guloso necessitou de **91,63 segundos** para concluir o processamento, sofrendo com o recalculo redundante de caminhos. Por outro lado, a Divisão e Conquista respondeu às mesmas 50.000 consultas em apenas **0,4438 segundos**, resultando em um ganho de velocidade superior a 200 vezes.

6 Conclusão

O trabalho cumpriu todos os objetivos propostos. A modelagem por Divisão e Conquista provou ser a escolha ideal para o sistema de navegação da mina, reduzindo a complexidade de tempo de dependência multiplicativa $O(Q \cdot HW)$ para um comportamento logarítmico em Q. A solução garante tempo de resposta sub-segundo para o volume total de operação da central.